

WealthWise — Project Report

Personal Net Worth, Loan & Expense Manager for the Indian market DBMS Experiential Learning — Level 3 Project · SQL + NoSQL (Option 1: PostgreSQL + MongoDB)

Team	<i>Member 1 name, SRN · Member 2 name, SRN</i>
Repository	<code>pranaava1tt-ally/wealth-management</code>
Stack	Next.js 14 (App Router) · TypeScript · PostgreSQL 16 · MongoDB 6+ · Tailwind CSS
Database objects	11 tables · 11 views · 14 functions · 1 stored procedure · 8 triggers · 35 indexes · 41 CHECK constraints · 13 foreign keys · 9 enum types · 4 MongoDB collections
Tests	67 automated tests (28 unit, 39 database integration), all passing

1. Problem statement

An Indian household's financial position is scattered across many places: a home loan with one bank and a vehicle loan with another, EPF and PPF accounts, gold held as jewellery and as sovereign gold bonds, mutual funds, fixed deposits, and a stream of UPI and card payments. Banking apps show one account at a time. Spreadsheets break as soon as an EMI is paid or a statement arrives.

As a result, people cannot easily answer basic questions:

- What am I actually worth today, and is that going up or down?
- How much of my home loan have I really repaid? (After seven years of a twenty-year loan, often less than a fifth of the principal.)
- If I put ₹5 lakh towards the loan, how much interest would I save?
- Where does my money go each month, and which costs are recurring?
- If the RBI raises the repo rate, what happens to my EMIs?

WealthWise brings assets, loans, repayment schedules and day-to-day transactions into one consistent database, and answers these questions with queries rather than hand calculation.

This is a data-management problem as much as a UI problem. Loans and their amortisation schedules are tightly constrained, highly relational data where correctness matters (a missed rupee compounds over 240 installments). Bank statement imports, receipts and news articles are irregular, semi-structured data. The project handles each kind with the database paradigm that suits it.

2. Objectives

1. Design a normalised relational schema for assets, loans, EMI schedules and transactions, with integrity enforced **in the database** through keys, CHECK constraints, enum types and triggers, not only in application code.
2. Implement financial logic (EMI amortisation, prepayment, net worth, a derived credit score) as **stored functions, procedures and triggers**, so that every calculation is visible, explainable and identical wherever it is used.

3. Use **MongoDB** for data that is genuinely schema-flexible (raw statement rows, uploaded documents, per-asset-type valuation metadata, cached news) and justify that choice rather than using it as a dumping ground.
4. Show **atomicity** where it matters: EMI payments, bulk statement imports and loan prepayments either complete entirely or not at all.
5. Provide a **role-based** Next.js web application (Individual and Advisor) with dashboards, charts, search, filtering, reports and a downloadable PDF.
6. Back every claim with **automated tests** against a real database.

3. Scope

In scope

- Asset, loan and transaction management for individuals, with an advisor role that can view assigned clients.
- Automatic reducing-balance EMI schedules, payment tracking, overdue detection and in-app reminders.
- Bank statement CSV import (HDFC, ICICI, SBI, Axis and Kotak layouts) with deduplication and rule-based categorisation; receipt OCR.
- Analytics: net worth trend, asset allocation, cash flow, recurring expenses, loan payoff progress, financial-year (April–March) reports with indicative Indian tax deductions.
- Smart features: prepayment simulation and application, what-if net worth projection, a derived credit score, a loan-product recommendation engine, and a news feed showing how rate changes affect the user's own floating-rate loans.

Out of scope, simulated deliberately

Real-world dependency	What WealthWise does instead
Credit bureau (CIBIL, Experian, Equifax)	A 300–900 score derived only from this database's repayment records, labelled as such everywhere it appears
Payment gateway or bank mandate	"Mark EMI paid" and "apply prepayment" record the event in the ledger; no money moves
Account Aggregator (RBI AA framework)	The user uploads a CSV statement they downloaded themselves
Market data feeds	Asset values are user-entered, with the history retained
Lender partnerships	A seeded catalogue of publicly advertised rate bands
Email or SMS delivery	Reminders are in-app only

WealthWise is a record-keeping and analysis tool. It gives no financial advice and stores no real account credentials.

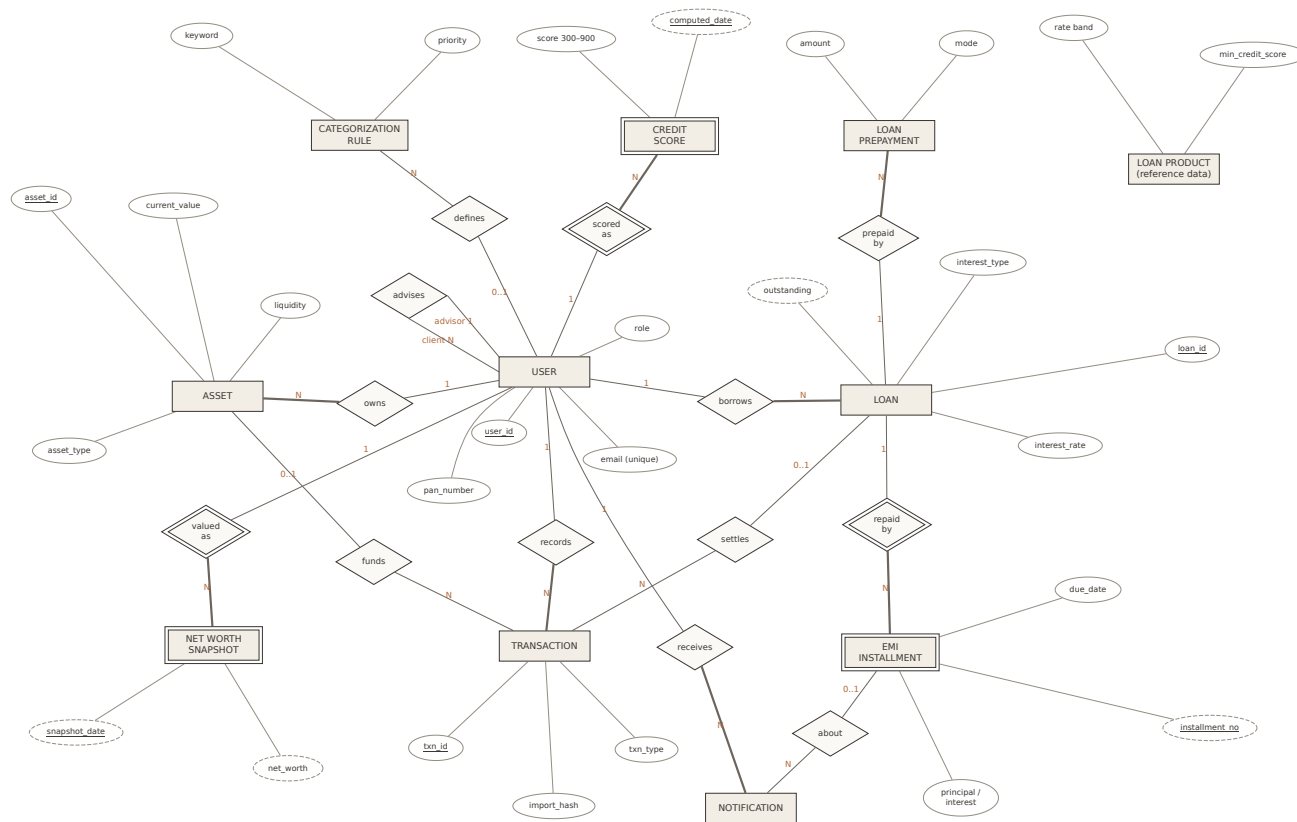
4. Functional requirements

ID	Requirement	Phase	Implemented by
FR-1	Sign up and log in; passwords hashed; JWT session; two roles	1	<code>auth-service.ts</code> , <code>bcrypt</code> (12 rounds), <code>httpOnly</code> JWT cookie

ID	Requirement	Phase	Implemented by
FR-2	Create, read, update and delete assets, with liquidity and valuation history	1	<code>assets</code> table, <code>asset_valuation_history</code> (Mongo)
FR-3	Create loans; the full EMI schedule is generated automatically	1	<code>fn_generate_emi_schedule()</code> inside one transaction
FR-4	Mark an EMI paid, atomically, with a ledger entry and auto-close	1	<code>payInstallment()</code> transaction
FR-5	Record income and expenses manually	1	<code>transactions</code> table
FR-6	Net worth dashboard with a historical trend	2	<code>net_worth_snapshots</code> , trigger, <code>v_net_worth_summary</code>
FR-7	Asset allocation chart	2	<code>v_asset_allocation</code> (window function)
FR-8	Loan payoff progress per loan	2	<code>v_loan_payoff_progress</code>
FR-9	Net worth refreshed automatically when data changes	2	<code>trg_fn_refresh_net_worth</code> on 4 tables
FR-10	Upcoming and overdue EMI view	2	<code>v_upcoming_emi_dues</code> , <code>fn_mark_overdue_emis()</code>
FR-11	Bank CSV import: staged, deduplicated, bulk-inserted atomically	3	<code>raw_imports</code> (Mongo) → one-transaction <code>UNNEST</code> insert
FR-12	Receipt photo → extracted merchant, amount and date → expense	3	Tesseract OCR (server-side) + confirm step
FR-13	Automatic categorisation with user overrides	3	<code>categorization_rules</code> , <code>fn_categorize()</code>
FR-14	Document vault: upload, tag and search	3	<code>documents</code> (Mongo) + <code>\$unwind</code> aggregation
FR-15	Prepayment simulator, with both modes compared	4	<code>fn_simulate_prepayment()</code> , <code>sp_apply_prepayment</code>
FR-16	What-if projection if a loan is closed early	4	<code>fn_project_net_worth()</code>
FR-17	Derived credit score, recomputed automatically	4	<code>fn_compute_credit_score()</code> + statement-level triggers
FR-18	Loan product recommendations	4	<code>fn_recommend_loan_products()</code>
FR-19	News feed with "how does this affect my loans?"	4	<code>news_articles</code> (Mongo) × floating-rate loans (Postgres)
FR-20	Financial-year reporting (April–March)	5	<code>v_fy_*</code> views, expression index
FR-21	Downloadable net worth PDF statement	5	<code>statement-pdf.ts</code> (pdfkit)
FR-22	EMI reminders	5	<code>notifications</code> , <code>fn_generate_emi_reminders()</code>
FR-23	Ledger full-text search	3	Generated <code>tsvector</code> column + GIN index
FR-24	Advisor view of assigned clients	4	<code>/advisor</code> , role check before any query

5. ER diagram

The conceptual model in **Chen notation**: rectangles are entities, diamonds are relationships, ellipses are attributes. A **double rectangle** is a weak entity and a **double diamond** its identifying relationship. An **underlined attribute** is a key and a **dashed underline** a partial key. A **dashed ellipse** is a derived attribute. A **thick line** marks total participation.



Source: docs/diagrams/er-conceptual.dot , rendered by npm run docs:generate .

Modelling decisions

Weak entities. An *EMI installment* has no identity outside its loan. "Installment 87" means nothing until you know which loan. It is identified by the owning loan plus the partial key `installment_no`, through the identifying relationship *repaid by*. *Net worth snapshot* (user + `snapshot_date`) and *credit score* (user + `computed_date`) are weak in the same way: one per user per day.

Recursive relationship. *Advises* relates USER to USER. An advisor (role `advisor`) advises N individual users, and each individual has at most one advisor.

Derived attributes. `outstanding` (on LOAN) and `net_worth` (on the snapshot) are derived. Outstanding is never stored; it is computed by `fn_loan_outstanding()` from the last paid installment. Net worth is stored in snapshots on purpose (see §6.3), and is always recomputed by trigger rather than written by hand.

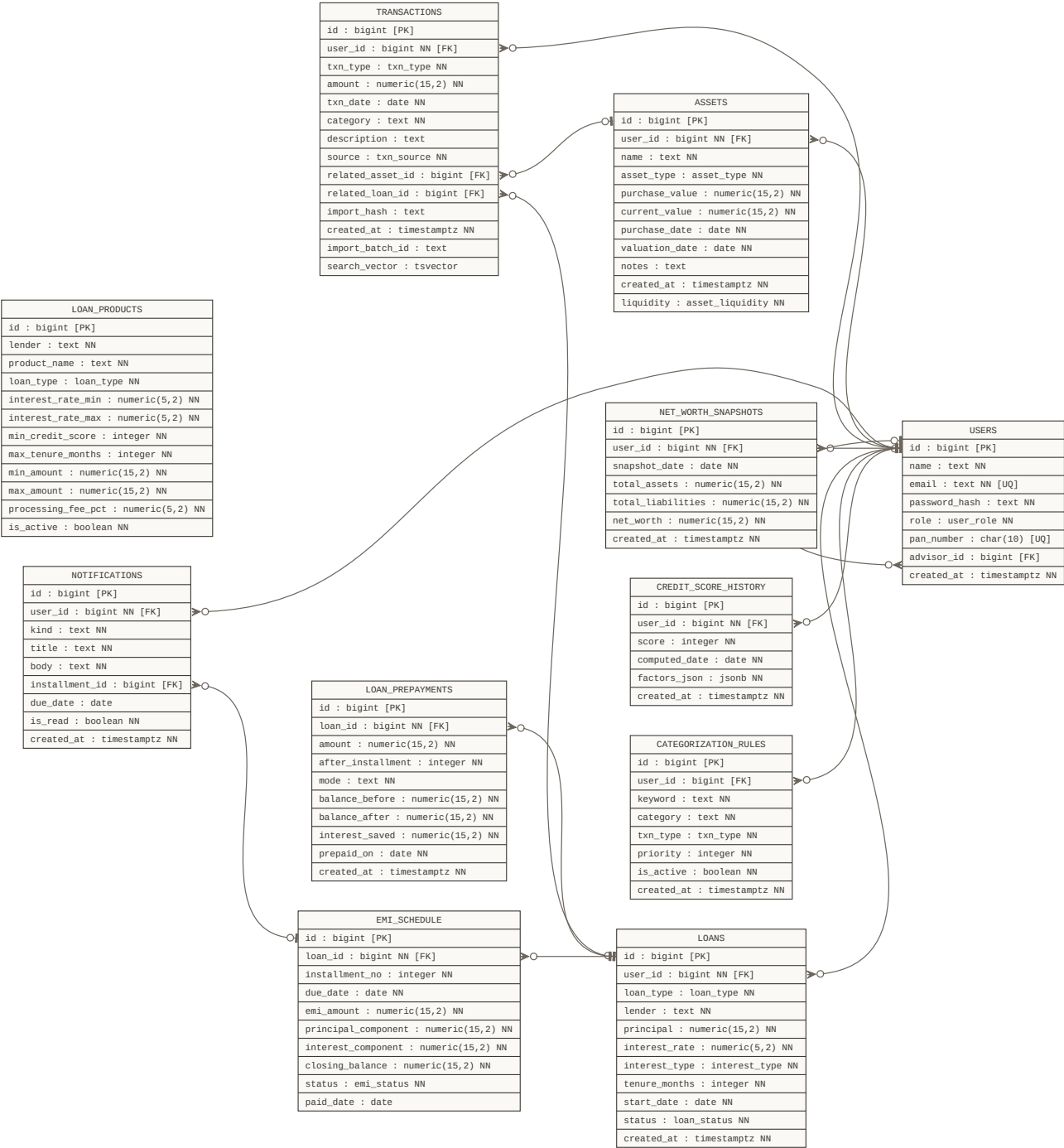
Optional relationships. A transaction *may* be linked to the asset that funded it (*funds*) or the loan it settled (*settles*), shown as 0..1. Deleting the asset or loan sets the link to NULL instead of deleting the ledger history.

Reference data. LOAN PRODUCT has no stored relationship to users. The recommendation engine matches products to a user at query time, so the link is computed, not persisted.

6. Relational schema

6.1 Schema diagram

Every table, column, type and foreign key, generated from the live database catalogue (so it cannot drift from the migrations). Crow's-foot notation: **>0** marks *zero or more* on the referencing side, and **||** marks *exactly one* on the referenced side.



6.2 ER to relational mapping

ER construct	Mapped to	Example
Strong entity	A table with a surrogate key <code>id</code> <code>BIGINT GENERATED ALWAYS AS IDENTITY</code>	<code>users</code> , <code>assets</code> , <code>loans</code>
1:N relationship	FK on the N side	<code>assets.user_id → users.id</code>
Total participation on the N side	FK declared <code>NOT NULL</code>	Every asset must have an owner
Partial participation	Nullable FK	<code>transactions.related_loan_id</code>
Weak entity	Table with the owner's FK plus a <code>UNIQUE (owner, partial key)</code> constraint, and a surrogate <code>id</code> for convenience	<code>emi_schedule</code> <code>UNIQUE (loan_id, installment_no)</code>
Identifying relationship	<code>ON DELETE CASCADE</code> from the owner	Deleting a loan deletes its schedule
Recursive 1:N	Self-referencing FK	<code>users.advisor_id → users.id</code>
Derived attribute	Function or view, not a column	<code>fn_loan_outstanding()</code> , <code>v_net_worth_summary</code>
Attribute with a fixed domain	PostgreSQL <code>ENUM</code> type	<code>loan_type</code> , <code>emi_status</code> , <code>txn_type</code>
Multi-valued attribute	MongoDB array, where queried	<code>documents.tags[]</code>

The schema in text form:

<code>users</code>	(<code>id</code> PK, <code>name</code> , <code>email</code> UQ, <code>password_hash</code> , <code>role</code> , <code>pan_number</code> UQ, <code>advisor_id</code> FK- <code>users</code> , <code>created_at</code>)
<code>assets</code>	(<code>id</code> PK, <code>user_id</code> FK- <code>users</code> , <code>name</code> , <code>asset_type</code> , <code>purchase_value</code> , <code>current_value</code> , <code>purchase_date</code> , <code>valuation_date</code> , <code>notes</code> , <code>liquidity</code> , <code>created_at</code>)
<code>loans</code>	(<code>id</code> PK, <code>user_id</code> FK- <code>users</code> , <code>loan_type</code> , <code>lender</code> , <code>principal</code> , <code>interest_rate</code> , <code>interest_type</code> , <code>tenure_months</code> , <code>start_date</code> , <code>status</code> , <code>created_at</code>)
<code>emi_schedule</code>	(<code>id</code> PK, <code>loan_id</code> FK- <code>loans</code> , <code>installment_no</code> , <code>due_date</code> , <code>emi_amount</code> , <code>principal_component</code> , <code>interest_component</code> , <code>closing_balance</code> , <code>status</code> , <code>paid_date</code> , UQ(<code>loan_id</code> , <code>installment_no</code>))
<code>loan_prepayments</code>	(<code>id</code> PK, <code>loan_id</code> FK- <code>loans</code> , <code>amount</code> , <code>after_installment</code> , <code>mode</code> , <code>balance_before</code> , <code>balance_after</code> , <code>interest_saved</code> , <code>prepaid_on</code> , <code>created_at</code>)
<code>transactions</code>	(<code>id</code> PK, <code>user_id</code> FK- <code>users</code> , <code>txn_type</code> , <code>amount</code> , <code>txn_date</code> , <code>category</code> , <code>description</code> , <code>source</code> , <code>related_asset_id</code> FK- <code>assets</code> , <code>related_loan_id</code> FK- <code>loans</code> , <code>import_hash</code> , <code>import_batch_id</code> , <code>search_vector</code> GENERATED, <code>created_at</code>)
<code>net_worth_snapshots</code>	(<code>id</code> PK, <code>user_id</code> FK- <code>users</code> , <code>snapshot_date</code> , <code>total_assets</code> , <code>total_liabilities</code> , <code>net_worth</code> , <code>created_at</code> , UQ(<code>user_id</code> , <code>snapshot_date</code>))
<code>credit_score_history</code>	(<code>id</code> PK, <code>user_id</code> FK- <code>users</code> , <code>score</code> , <code>computed_date</code> , <code>factors_json</code> , <code>created_at</code> , UQ(<code>user_id</code> , <code>computed_date</code>))
<code>loan_products</code>	(<code>id</code> PK, <code>lender</code> , <code>product_name</code> , <code>loan_type</code> , <code>interest_rate_min</code> , <code>interest_rate_max</code> , <code>min_credit_score</code> , <code>max_tenure_months</code> , <code>min_amount</code> , <code>max_amount</code> , <code>processing_fee_pct</code> , <code>is_active</code> , UQ(<code>lender</code> , <code>product_name</code>))
<code>categorization_rules</code>	(<code>id</code> PK, <code>user_id</code> FK- <code>users</code> NULL, <code>keyword</code> , <code>category</code> , <code>txn_type</code> , <code>priority</code> , <code>is_active</code> , <code>created_at</code>)
<code>notifications</code>	(<code>id</code> PK, <code>user_id</code> FK- <code>users</code> , <code>kind</code> , <code>title</code> , <code>body</code> , <code>installment_id</code> FK- <code>emi_schedule</code> , <code>due_date</code> , <code>is_read</code> , <code>created_at</code>)

6.3 Normalisation

Every table is in **third normal form**, and all except the deliberate cases below are in **BCNF**. The only determinants are keys: a loan's terms depend on the loan id alone, and an installment's figures on (`loan_id`, `installment_no`) alone.

There are three deliberate exceptions, each kept for a stated reason:

Stored derived data	Why it is kept	How it stays correct
<code>net_worth_snapshots.net_worth</code> , <code>total_assets</code> , <code>total_liabilities</code>	It is a historical record . Yesterday's net worth cannot be recomputed from today's asset values once those values have changed.	Written only by <code>fn_refresh_net_worth()</code> , called from triggers; <code>net_worth</code> equals assets minus liabilities by construction
<code>emi_schedule.emi_amount</code> , <code>closing_balance</code>	The amortisation table is a contract : once an installment is paid, its figures must never change, even if the formula or the loan is later edited	Generated by one function; a paid schedule is locked against regeneration (<code>WW409</code>)
<code>transactions.search_vector</code>	Full-text search needs an indexable <code>tsvector</code>	A <code>STORED</code> generated column, so the database maintains it and it cannot drift

7. Database choice and justification

The brief asks that each database be chosen for the nature of the data, "not merely for technology demonstration". Here is the split, and why.

PostgreSQL — the system of record

Users, assets, loans, the EMI schedule, transactions, snapshots, scores, products, rules and notifications.

- **The data is relational and tightly constrained.** A loan has exactly one owner; an installment belongs to exactly one loan; a payment must reference a real installment. These are foreign keys, and violating them is a bug the database should refuse.
- **Correctness depends on transactions.** Paying an EMI updates the schedule, writes the ledger and may close the loan. A prepayment rewrites a schedule and debits an asset. Both need ACID guarantees.
- **The arithmetic belongs next to the data.** `NUMERIC(15,2)` gives exact rupee-and-paise arithmetic (no floating-point drift across 240 installments), and `plpgsql` lets the EMI formula live in one place.
- **Analytics are relational queries.** Allocation, cash flow, recurring expenses and FY reports are `GROUP BY`, window-function and `JOIN` queries.

MongoDB — schema-flexible documents

Collection	Why it is a document, not a row
<code>raw_imports</code>	Each bank's CSV has different columns. The raw rows are kept verbatim for audit, before normalisation. Forcing them into one table would mean either losing columns or a wide, mostly-NULL table.
<code>asset_valuation_history</code>	What matters differs by asset class: gold has grams, purity and rate per gram; property has square feet, city and circle rate; equity has units and NAV. One document per valuation fits; the relational alternatives are a sparse table or an EAV design.
<code>documents</code>	Uploaded files carry irregular metadata (survey numbers on a deed, a GSTIN on a receipt) and a free-form <code>tags[]</code> array that is queried directly.

Collection	Why it is a document, not a row
news_articles	Feed items are semi-structured; rate fields are present on some articles and absent on most.

The two databases are linked deliberately. `transactions.import_batch_id` holds the Mongo `ObjectId` of the staged import. Postgres cannot enforce a foreign key across that boundary, which is documented in a column comment. The news feature runs a **cross-database query**: the rate change comes from a Mongo document, and the affected loans come from Postgres.

Why not a vector database

§3 of the brief offers SQL + NoSQL **or** SQL + vector; this project takes the first. §8 recommends embeddings for unstructured, text-heavy data. WealthWise's data is overwhelmingly numeric and relational: amounts, rates, dates and schedules. Its one text-heavy collection, cached news, is served well by category and tag filters. Adding embeddings would be technology for its own sake, which §18 warns against.

8. Data dictionary

The full data dictionary is generated from the live catalogue: every column's type, nullability, default and key role, plus every CHECK constraint and every index with its definition.

→ [docs/DATA_DICTIONARY.md](#)

Summary of what the database enforces:

Rule	Mechanism
All money is exact	<code>NUMERIC(15,2)</code> everywhere, never <code>float</code>
Amounts are positive; direction is in <code>txn_type</code>	<code>CHECK (amount > 0)</code>
Rates are sane	<code>CHECK (interest_rate > 0 AND interest_rate <= 60)</code>
PAN is well-formed	<code>CHECK (pan_number ~ '^[A-Z]{5}[0-9]{4}[A-Z]\$')</code>
Emails are lowercase and unique	<code>CHECK (email = lower(email)) + UNIQUE</code>
A paid installment has a paid date, an unpaid one does not	Multi-column <code>CHECK emi_paid_date_consistency</code>
Asset valued no earlier than purchase, and not in the future	<code>CHECK</code> constraints, the second added via <code>ALTER</code>
Prepayment arithmetic adds up	<code>CHECK (balance_after = balance_before - amount)</code>
One snapshot / one score per user per day	<code>UNIQUE (user_id, snapshot_date), UNIQUE (user_id, computed_date)</code>
No duplicate imported transactions	Partial unique index on <code>(user_id, import_hash)</code>
No duplicate system rules (NULL ≠ NULL)	Partial unique index on <code>upper(keyword) WHERE user_id IS NULL</code>
Payment history is never erased	<code>fn_generate_emi_schedule</code> raises <code>WW409</code> once anything is paid

9. Database implementation

Eight migrations, applied in order by a transactional runner (`scripts/migrate.ts`). Each file runs in its own transaction and is recorded in `schema_migrations` .

Migration	Contents
001_core_schema	8 tables, 8 enum types, keys, CHECK constraints, cascade rules
002_indexes	Indexes, each annotated with the query that justifies it
003_emi_functions	<code>fn_calculate_emi</code> , <code>fn_generate_emi_schedule</code> , <code>fn_loan_outstanding</code>
004_networth_trigger	<code>fn_refresh_net_worth</code> + one trigger body on three tables
005_alters_and_views	<code>ALTER TABLE ... ADD COLUMN liquidity</code> , <code>ADD CONSTRAINT</code> , <code>DROP/CREATE TRIGGER</code> ; 7 reporting views
006_import_and_search	Categorisation rules + <code>fn_categorize</code> ; cross-DB <code>import_batch_id</code> ; generated <code>tsvector</code> + GIN
007_fix_recategory	Fix found by testing (see §13.3)
008_smart_features	Prepayment (function + procedure), credit score + statement triggers, recommendations, projection, FY views + expression index, notifications

9.1 DDL coverage

Statement	Where
<code>CREATE TABLE / TYPE / INDEX / VIEW / FUNCTION / PROCEDURE / TRIGGER</code>	Throughout
<code>ALTER TABLE ... ADD COLUMN</code>	<code>assets.liquidity</code> , <code>transactions.import_batch_id</code> , <code>transactions.search_vector</code>
<code>ALTER TABLE ... ADD / DROP CONSTRAINT</code>	<code>assets_valuation_not_in_future</code>
<code>DROP TRIGGER / DROP SCHEMA</code>	Trigger narrowing in 005 and 008; <code>migrate --reset</code>

9.2 Advanced features (the brief asks for at least four)

#	Feature	Implementation
1	Transactions	EMI payment, loan creation and schedule, bulk CSV import, prepayment: each is one atomic unit with rollback tested
2	Stored functions	14, including EMI, amortisation, prepayment simulation, credit score, recommendations, projection, categorisation
3	Stored procedure	<code>sp_apply_prepayment</code> , called with <code>CALL</code> , using row locks (<code>FOR UPDATE</code>) and an <code>INOUT</code> parameter
4	Triggers	Row-level net worth refresh on 4 tables; statement-level triggers with transition tables for the credit score
5	Views	11 reporting views: net worth, dues, allocation, payoff, cash flow, category spend, recurring expenses (HAVING), 4 FY views
6	Indexing	35 indexes, including partial , partial unique , expression (on an IMMUTABLE function) and GIN indexes
7	Full-text search	Generated <code>tsvector</code> column, weighted A/B, GIN index, <code>websearch_to_tsquery</code>

#	Feature	Implementation
8	NoSQL aggregation	<code>\$group</code> , <code>\$unwind</code> , <code>\$project</code> and <code>\$sort</code> pipelines over imports, documents and news
9	Window functions	<code>SUM(SUM(...)) OVER (PARTITION BY ...)</code> , <code>LAG()</code> , running totals

9.3 Index justification (selection)

Index	Type	Query it serves
<code>idx_emi_due_pending</code>	Partial , WHERE status IN ('pending', 'overdue')	The upcoming-dues query. Paid rows, the vast majority over time, are excluded, so the index stays small.
<code>idx_txn_import_hash</code>	Partial unique	Makes import deduplication a database guarantee; manual rows (NULL hash) are unconstrained
<code>idx_catrule_system_keyword</code>	Partial unique on <code>upper(keyword)</code>	Stops duplicate system rules, which a plain <code>UNIQUE</code> misses because <code>NULL ≠ NULL</code>
<code>idx_txn_user_fy</code>	Expression on <code>fn_fy_start_year(txn_date)</code>	FY reports filter on the computed year, which an index on <code>txn_date</code> cannot serve. Verified with <code>EXPLAIN</code> .
<code>idx_txn_search</code>	GIN on <code>search_vector</code>	Full-text ledger search. Verified with <code>EXPLAIN</code> .
<code>idx_txn_user_date</code>	Composite (<code>user_id</code> , <code>txn_date</code> DESC)	The ledger, newest first: filter and sort from one index
<code>idx_notif_unread</code>	Partial, WHERE NOT is_read	The bell badge count

9.4 MongoDB data model

```

raw_imports      { _id, userId, sourceBank, fileName, fileType, rawRows: [...], rowCount,
                  parsedAt, matchStatus: 'staged'|'committed'|'reverted', committedAt, insertedCount }
asset_valuation_history { userId, assetId, valuationDate, value,
                        typeSpecificMetadata: { grams, purity, ratePerGram } | { sqft, city } | ... }
documents        { _id, userId, refType: 'loan'|'asset'|'general', refId, title, fileName,
                  originalName, fileUrl, mimeType, sizeBytes, checksum, tags: [ ... ], uploadedAt }
news_articles    { _id, headline, source, category, tags: [ ... ], body, url, publishedAt,
                  relatedRateType, rateChangeBps, isSample, fetchedAt }

```

Retrieval strategy. Every document query filters on `userId` first. Imports are listed by `parsedAt` descending. Documents are searched by an escaped, case-insensitive regex over title, filename and tags. News is cached with a 6-hour TTL and refreshed lazily on read (or by a cron hitting `POST /api/news/refresh`), upserting by article URL so re-fetches never duplicate.

Aggregation pipelines:

```

// Vault: count each tag independently – $unwind flattens the tags array
[ { $match: { userId } }, { $unwind: '$tags' },
  { $group: { _id: '$tags', count: { $sum: 1 } } }, { $sort: { count: -1 } } ]

// Imports: per-bank statistics
[ { $match: { userId } },
  { $group: { _id: '$sourceBank', imports: { $sum: 1 }, totalRows: { $sum: '$rowCount' },
            committed: { $sum: { $cond: [ { $eq: [ '$matchStatus', 'committed' ] }, 1, 0 ] } } } } ]

```

10. Important queries

Every query below is in the repository verbatim. The application has no ORM, so these are the actual statements it runs.

Q1. EMI by the reducing-balance formula (`fn_calculate_emi`)

$E = P \cdot r \cdot (1+r)^n / ((1+r)^n - 1)$, where $r = \text{annual rate} / 12 / 100$.

```
v_monthly_rate := p_annual_rate / 12 / 100;
v_growth      := POWER(1 + v_monthly_rate, p_tenure_months);
RETURN ROUND(p_principal * v_monthly_rate * v_growth / (v_growth - 1), 2);
```

₹50,00,000 at 8.50% over 240 months gives **₹43,391.16**, and the generated principal components sum to exactly ₹50,00,000.00 (\$13).

Q2. Asset allocation: GROUP BY with a window over the aggregate (`v_asset_allocation`)

```
SELECT user_id, asset_type, SUM(current_value) AS current_value,
       ROUND(100.0 * SUM(current_value)
             / NULLIF(SUM(SUM(current_value)) OVER (PARTITION BY user_id, 0), 2) AS allocation_pct
FROM assets GROUP BY user_id, asset_type;
```

`SUM(SUM(...)) OVER` is each user's grand total, evaluated *after* grouping. A plain subquery cannot do that in the same pass.

Q3. Recurring expenses: HAVING on an aggregate (`v_recurring_expenses`)

```
SELECT user_id, category, COUNT(DISTINCT date_trunc('month', txn_date)) AS months_seen,
       ROUND(SUM(amount) / NULLIF(COUNT(DISTINCT date_trunc('month', txn_date)), 0), 2) AS monthly_run_rate
FROM transactions
WHERE txn_type = 'expense' AND txn_date >= CURRENT_DATE - INTERVAL '12 months'
GROUP BY user_id, category
HAVING COUNT(DISTINCT date_trunc('month', txn_date)) >= 3;
```

`WHERE` cannot express "appeared in three months", because `WHERE` runs before grouping.

Q4. Loan summary: one pass with FILTER (`v_loan_payoff_progress`, excerpt)

```
SELECT l.id, COUNT(e.id) FILTER (WHERE e.status = 'paid')           AS paid_installments,
       SUM(e.interest_component) FILTER (WHERE e.status = 'paid') AS interest_paid,
       MIN(e.due_date) FILTER (WHERE e.status <> 'paid')           AS next_due_date,
       fn_loan_outstanding(l.id)                                   AS outstanding
FROM loans l LEFT JOIN emi_schedule e ON e.loan_id = l.id GROUP BY l.id;
```

Q5. Atomic bulk import (`commitImport`, run inside `BEGIN ... COMMIT`)

```
INSERT INTO transactions (user_id, txn_type, amount, txn_date, category, description, source, import_hash,
import_batch_id)
SELECT $1, t.txn_type::txn_type, t.amount, t.txn_date::date, t.category, t.description, 'import', t.import_hash,
$8
FROM UNNEST($2::text[], $3::numeric[], $4::text[], $5::text[], $6::text[], $7::text[])
      AS t(txn_type, amount, txn_date, category, description, import_hash)
ON CONFLICT (user_id, import_hash) WHERE import_hash IS NOT NULL DO NOTHING
RETURNING txn_type, amount;
```

One statement for the whole statement file, with deduplication guaranteed by the partial unique index rather than by application logic.

Q6. Categorisation with precedence in ORDER BY (fn_categorize)

```
SELECT r.category, r.txn_type, r.keyword FROM categorization_rules r
WHERE r.is_active AND (r.user_id = p_user_id OR r.user_id IS NULL)
  AND upper(p_description) LIKE '%' || upper(r.keyword) || '%'
ORDER BY (r.user_id IS NOT NULL) DESC, r.priority, length(r.keyword) DESC
LIMIT 1;
```

Q7. Statement-level trigger with transition tables (credit score)

```
CREATE TRIGGER trg_emi_credit_score AFTER UPDATE ON emi_schedule
REFERENCING OLD TABLE AS old_rows NEW TABLE AS new_rows
FOR EACH STATEMENT EXECUTE FUNCTION trg_fn_credit_score_emi();
-- body: FOR r IN SELECT DISTINCT l.user_id FROM new_rows n JOIN old_rows o ON o.id = n.id
--       JOIN loans l ON l.id = n.loan_id WHERE o.status IS DISTINCT FROM n.status ...
--       LOOP PERFORM fn_compute_credit_score(r.user_id); END LOOP;
```

Marking 24 EMIs paid in one UPDATE rescores the user **once**, not 24 times. This is tested.

Q8. Prepayment as a stored procedure, with row locks (sp_apply_prepayment, outline)

```
SELECT l.lender INTO v_lender FROM loans l
WHERE l.id = p_loan_id AND l.user_id = p_user_id AND l.status = 'active' FOR UPDATE;
UPDATE assets SET current_value = current_value - p_amount WHERE id = p_from_asset_id;
SELECT array_agg(s ORDER BY s.installment_no) INTO v_rows FROM fn_simulate_prepayment(p_loan_id, p_amount, p_mode)
s;
DELETE FROM emi_schedule WHERE loan_id = p_loan_id AND status <> 'paid';
INSERT INTO emi_schedule (...) SELECT ... FROM unnest(v_rows) r;
INSERT INTO loan_prepayments (...) VALUES (...); -- CHECK verifies the arithmetic
INSERT INTO transactions (...) VALUES (...);
```

Called as CALL sp_apply_prepayment(\$1,\$2,\$3,\$4,\$5,NULL). Net worth is unchanged afterwards, because the asset and the liability fall by the same amount. That is tested too.

Q9. Recommendation engine: CROSS JOIN profile x catalogue (fn_recommend_loan_products, excerpt)

```
WITH profile AS (SELECT <latest score>, <avg monthly income>, <existing EMI burden>),
  priced AS (SELECT p.*, pr.*, <rate interpolated in band by score> AS est_rate
FROM loan_products p CROSS JOIN profile pr WHERE p.is_active AND p.loan_type = $2)
SELECT ..., (score >= min_credit_score AND amount BETWEEN min_amount AND max_amount
  AND (existing_emi + emi) / monthly_income <= 0.5) AS eligible,
  40*rate_term + 25*score_headroom + 15*fee_term + 20*affordability AS match_score
FROM costed ORDER BY eligible DESC, match_score DESC;
```

Q10. Advisor triage: JOIN + LATERAL + correlated subquery (advisorClients)

```
SELECT u.id, u.name, s.net_worth, s.debt_to_asset_pct, cs.score,
  (SELECT COUNT(*) FROM v_upcoming_emi_dues d WHERE d.user_id = u.id AND d.urgency = 'overdue') AS overdue
FROM users u
JOIN v_net_worth_summary s ON s.user_id = u.id
LEFT JOIN LATERAL (SELECT c.score FROM credit_score_history c
  WHERE c.user_id = u.id ORDER BY c.computed_date DESC LIMIT 1) cs ON TRUE
WHERE u.advisor_id = $1 ORDER BY overdue DESC, u.name;
```

Q11. Full-text search ranked by relevance (listTransactions)

```
SELECT ... FROM transactions
WHERE user_id = $1 AND ($6::text IS NULL OR search_vector @@ websearch_to_tsquery('english', $6))
ORDER BY CASE WHEN $6::text IS NULL THEN 0
  ELSE ts_rank(search_vector, websearch_to_tsquery('english', $6)) END DESC, txn_date DESC;
```

Q12. Cross-database query: news × floating-rate loans (rateImpact)

The rate change in basis points comes from a MongoDB article, then:

```
SELECT l.lender, fn_calculate_emi(o.outstanding, l.interest_rate, r.remaining::int) AS old_emi,
      fn_calculate_emi(o.outstanding, l.interest_rate + $2/100.0, r.remaining::int) AS new_emi
FROM loans l
CROSS JOIN LATERAL (SELECT fn_loan_outstanding(l.id) AS outstanding) o
CROSS JOIN LATERAL (SELECT COUNT(*) AS remaining FROM emi_schedule e
                    WHERE e.loan_id = l.id AND e.status <> 'paid') r
WHERE l.user_id = $1 AND l.status = 'active' AND l.interest_type = 'floating';
```

11. System architecture



- **Layering is strict.** No React component issues a query, and no route handler contains inline SQL.
- **No ORM.** Postgres is reached through `pg` with parameterised SQL, so every query in §10 is visible and explainable as written.
- **Errors carry status from the database.** Stored routines raise SQLSTATEs of the form `WWnnnn`, and the API maps `WW409` to HTTP 409, `WW404` to 404 and so on, so business-rule violations reach the user as clear messages rather than 500s.
- **Mongo is optional at runtime.** If it is unreachable, imports still complete (without the raw audit copy), the news feed serves labelled sample data, and the vault shows an explicit "MongoDB not reachable" panel. A failed connection is cached for 60 seconds so requests do not stall.

Security

Concern	Measure
Passwords	bcrypt, 12 rounds; never logged or returned
Sessions	HS256 JWT in an <code>httpOnly</code> , <code>sameSite=lax</code> cookie, so it cannot be read by script
Account enumeration	Identical message and comparable timing for "no such user" and "wrong password"
Authorisation	Ownership in the <code>WHERE</code> clause (<code>... AND user_id = \$2</code>), with no check-then-act gap; advisors limited to assigned clients
SQL injection	Every query parameterised; optional filters use <code>(\$n IS NULL OR col = \$n)</code> instead of string building
Uploads	MIME allowlist, size limits, generated filenames (never user-supplied paths)

Concern	Measure
Secrets	All credentials from environment variables; <code>.env</code> is gitignored
PII	PAN is masked on the PDF statement
Privacy	Receipt OCR runs on the server with a bundled model; images never leave it

12. Application screenshots

WEALTHWISE

Net worth, loans and expenses — in one ledger.

EMAIL

PASSWORD

Sign in

Don't have an account? Sign up

Login

WEALTHWISE

Dashboard

Assets

Loans

Ledger

Import

Vault

Insights

Reports

News

Priya Sharma

Light

Sign out

Overview

NET WORTH

₹1.06 Cr

11 assets · 2 active loans

TOTAL ASSETS

₹1.54 Cr

₹38.80 L Liquid

OUTSTANDING DEBT

₹47.63 L

55.9% of assets

MONTHLY EMI BURDEN

₹55,391.42

₹130 L income this month

NET WORTH TREND

₹1.00

₹1.20

₹1.40

₹1.60

₹1.80

₹2.00

₹2.20

₹2.40

₹2.60

₹2.80

₹3.00

₹3.20

₹3.40

₹3.60

₹3.80

₹4.00

₹4.20

₹4.40

₹4.60

₹4.80

₹5.00

₹5.20

₹5.40

₹5.60

₹5.80

₹6.00

₹6.20

₹6.40

₹6.60

₹6.80

₹7.00

₹7.20

₹7.40

₹7.60

₹7.80

₹8.00

₹8.20

₹8.40

₹8.60

₹8.80

₹9.00

₹9.20

₹9.40

₹9.60

₹9.80

₹10.00

₹10.20

₹10.40

₹10.60

₹10.80

₹11.00

₹11.20

₹11.40

₹11.60

₹11.80

₹12.00

₹12.20

₹12.40

₹12.60

₹12.80

₹13.00

₹13.20

₹13.40

₹13.60

₹13.80

₹14.00

₹14.20

₹14.40

₹14.60

₹14.80

₹15.00

₹15.20

₹15.40

₹15.60

₹15.80

₹16.00

₹16.20

₹16.40

₹16.60

₹16.80

₹17.00

₹17.20

₹17.40

₹17.60

₹17.80

₹18.00

₹18.20

₹18.40

₹18.60

₹18.80

₹19.00

₹19.20

₹19.40

₹19.60

₹19.80

₹20.00

₹20.20

₹20.40

₹20.60

₹20.80

₹21.00

₹21.20

₹21.40

₹21.60

₹21.80

₹22.00

₹22.20

₹22.40

₹22.60

₹22.80

₹23.00

₹23.20

₹23.40

₹23.60

₹23.80

₹24.00

₹24.20

₹24.40

₹24.60

₹24.80

₹25.00

₹25.20

₹25.40

₹25.60

₹25.80

₹26.00

₹26.20

₹26.40

₹26.60

₹26.80

₹27.00

₹27.20

₹27.40

₹27.60

₹27.80

₹28.00

₹28.20

₹28.40

₹28.60

₹28.80

₹29.00

₹29.20

₹29.40

₹29.60

₹29.80

₹30.00

₹30.20

₹30.40

₹30.60

₹30.80

₹31.00

₹31.20

₹31.40

₹31.60

₹31.80

₹32.00

₹32.20

₹32.40

₹32.60

₹32.80

₹33.00

₹33.20

₹33.40

₹33.60

₹33.80

₹34.00

₹34.20

₹34.40

₹34.60

₹34.80

₹35.00

₹35.20

₹35.40

₹35.60

₹35.80

₹36.00

₹36.20

₹36.40

₹36.60

₹36.80

₹37.00

₹37.20

₹37.40

₹37.60

₹37.80

₹38.00

₹38.20

₹38.40

₹38.60

₹38.80

₹39.00

₹39.20

₹39.40

₹39.60

₹39.80

₹40.00

₹40.20

₹40.40

₹40.60

₹40.80

₹41.00

₹41.20

₹41.40

₹41.60

₹41.80

₹42.00

₹42.20

₹42.40

₹42.60

₹42.80

₹43.00

₹43.20

₹43.40

₹43.60

₹43.80

₹44.00

₹44.20

₹44.40

₹44.60

₹44.80

₹45.00

₹45.20

₹45.40

₹45.60

₹45.80

₹46.00

₹46.20

₹46.40

₹46.60

₹46.80

₹47.00

₹47.20

₹47.40

₹47.60

₹47.80

₹48.00

₹48.20

₹48.40

₹48.60

₹48.80

₹49.00

₹49.20

₹49.40

₹49.60

₹49.80

₹50.00

₹50.20

₹50.40

₹50.60

₹50.80

₹51.00

₹51.20

₹51.40

₹51.60

₹51.80

₹52.00

₹52.20

₹52.40

₹52.60

₹52.80

₹53.00

₹53.20

₹53.40

₹53.60

₹53.80

₹54.00

₹54.20

₹54.40

₹54.60

₹54.80

₹55.00

₹55.20

₹55.40

₹55.60

₹55.80

₹56.00

₹56.20

₹56.40

₹56.60

₹56.80

₹57.00

₹57.20

₹57.40

₹57.60

₹57.80

₹58.00

₹58.20

₹58.40

₹58.60

₹58.80

₹59.00

₹59.20

₹59.40

₹59.60

₹59.80

₹60.00

₹60.20

₹60.40

₹60.60

₹60.80

₹61.00

₹61.20

₹61.40

₹61.60

₹61.80

₹62.00

₹62.20

₹62.40

₹62.60

₹62.80

₹63.00

₹63.20

₹63.40

₹63.60

₹63.80

₹64.00

₹64.20

₹64.40

₹64.60

₹64.80

₹65.00

₹65.20

₹65.40

₹65.60

₹65.80

₹66.00

₹66.20

₹66.40

₹66.60

₹66.80

₹67.00

₹67.20

₹67.40

₹67.60

₹67.80

₹68.00

₹68.20

₹68.40

₹68.60

₹68.80

₹69.00

₹69.20

₹69.40

₹69.60

₹69.80

₹70.00

₹70.20

₹70.40

₹70.60

₹70.80

₹71.00

₹71.20

₹71.40

₹71.60

₹71.80

₹72.00

₹72.20

₹72.40

₹72.60

₹72.80

₹73.00

₹73.20

₹73.40

₹73.60

₹73.80

₹74.00

₹74.20

₹74.40

₹74.60

₹74.80

₹75.00

₹75.20

₹75.40

₹75.60

₹75.80

₹76.00

₹76.20

₹76.40

₹76.60

₹76.80

₹77.00

₹77.20

₹77.40

₹77.60

₹77.80

₹78.00

₹78.20

₹78.40

₹78.60

₹78.80

₹79.00

₹79.20

₹79.40

₹79.60

₹79.80

₹80.00

₹80.20

₹80.40

₹80.60

₹80.80

₹81.00

₹81.20

₹81.40

₹81.60

₹81.80

₹82.00

₹82.20

₹82.40

₹82.60

₹82.80

₹83.00

₹83.20

₹83.40

₹83.60

₹83.80

₹84.00

₹84.20

₹84.40

₹84.60

₹84.80

₹85.00

₹85.20

₹85.40

₹85.60

₹85.80

₹86.00

₹86.20

₹86.40

₹86.60

₹86.80

₹87.00

₹87.20

₹87.40

₹87.60

₹87.80

₹88.00

₹88.20

₹88.40

₹88.60

₹88.80

₹89.00

₹89.20

₹89.40

₹89.60

₹89.80

₹90.00

₹90.20

₹90.40

₹90.60

₹90.80

₹91.00

₹91.20

₹91.40

₹91.60

₹91.80

₹92.00

₹92.20

₹92.40

₹92.60

₹92.80

₹93.00

₹93.20

₹93.40

₹93.60

₹93.80

₹94.00

₹94.20

₹94.40

₹94.60

₹94.80

₹95.00

₹95.20

₹95.40

₹95.60

₹95.80

₹96.00

₹96.20

₹96.40

₹96.60

₹96.80

₹97.00

₹97.20

₹97.40

₹97.60

₹97.80

₹98.00

₹98.20

₹98.40

₹98.60

₹98.80

₹99.00

₹99.20

₹99.40

₹99.60

₹99.80

₹100.00

₹100.20

₹100.40

₹100.60

₹100.80

₹101.00

₹101.20

₹101.40

₹101.60

₹101.80

₹102.00

₹102.20

₹102.40

₹102.60

₹102.80

₹103.00

₹103.20

₹103.40

₹103.60

₹103.80

₹104.00

₹104.20

₹104.40

₹104.60

₹104.80

₹105.00

₹105.20

₹105.40

₹105.60

₹105.80

₹106.00

₹106.20

₹106.40

₹106.60

₹106.80

₹107.00

₹107.20

₹107.40

₹107.60

₹107.80

₹108.00

₹108.20

₹108.40

₹108.60

₹108.80

₹109.00

₹109.20

₹109.40

₹109.60

₹109.80

₹110.00

₹110.20

₹110.40

₹110.60

₹110.80

₹111.00

₹111.20

₹111.40

₹111.60

₹111.80

₹112.00

₹112.20

₹112.40

₹112.60

₹112.80

₹113.00

₹113.20

₹113.40

₹113.60

₹113.80

₹114.00

₹114.20

₹114.40

₹114.60

₹114.80

₹115.00

₹115.20

₹115.40

₹115.60

₹115.80

₹116.00

₹116.20

₹116.40

₹116.60

₹116.80

₹117.00

₹117.20

₹117.40

₹117.60

₹117.80

₹118.00

₹118.20

₹118.40

₹118.60

₹118.80

₹119.00

₹119.20

₹119.40

₹119.60

₹119.80

₹120.00

₹120.20

₹120.40

₹120.60

₹120.80

₹121.00

₹121.20

₹121.40

₹121.60

₹121.80

₹122.00

₹122.20

₹122.40

₹122.60

₹122.80

₹123.00

₹123.20

₹123.40

₹123.60

₹123.80

₹124.00

₹124.20

₹124.40

₹124.60

₹124.80

₹125.00

₹125.20

₹125.40

₹125.60

₹125.80

₹126.00

₹126.20

₹126.40

₹126.60

₹126.80

₹127.00

₹127.20

₹127.40

₹127.60

₹127.80

₹128.00

₹128.20

₹128.40

₹128.60

₹128.80

₹129.00

₹129.20

₹129.40

₹129.60

₹129.80

₹130.00

₹130.20

₹130.40

₹130.60

₹130.80

₹131.00

₹131.20

₹131.40

₹131.60

₹131.80

₹132.00

₹132.20

₹132.40

₹132.60

₹132.80

₹133.00

₹133.20

₹133.40

₹133.60

₹133.80

₹134.00

₹134.20

₹134.40

₹134.60

₹134.80

₹135.00

₹135.20

₹135.40

₹135.60

₹135.80

₹136.00

₹136.20

₹136.40

₹136.60

₹136.80

₹137.00

₹137.20

₹137.40

₹137.60

₹137.80

₹138.00

₹138.20

₹138.40

₹138.60

₹138.80

₹139.00

₹139.20

₹139.40

₹139.60

₹139.80

₹140.00

₹140.20

₹140.40

₹140.60

₹140.80

₹141.00

₹141.20

₹141.40

₹141.60

₹141.80

₹142.00

₹142.20

₹142.40

₹142.60

₹142.80

₹143.00

₹143.20

₹143.40

₹143.60

₹143.80

₹144.00

₹144.20

₹144.40

₹144.60

₹144.80

₹145.00

₹145.20

₹145.40

₹145.60

₹145.80

₹146.00

₹146.20

₹146.40

₹146.60

₹146.80

₹147.00

₹147.20

₹147.40

₹147.60

₹147.80

₹148.00

₹148.20

₹148.40

₹148.60

₹148.80

₹149.00

₹149.20

₹149.40

₹149.60

₹149.80

₹150.00

₹150.20

₹150.40

₹150.60

₹150.80

₹151.00

₹151.20

₹151.40

₹151.60

₹151.80

₹152.00

₹152.20

₹152.40

₹152.60

₹152.80

₹153.00

₹153.20

₹153.40

₹153.60

₹153.80

₹154.00

₹154.20

₹154.40

₹154.60

₹154.80

₹155.00

₹155.20

₹155.40

₹155.60

₹155.80

₹156.00

₹156.20

₹156.40

₹156.60

₹156.80

₹157.00

₹157.20

₹157.40

₹157.60

₹157.80

₹158.00

₹158.20

₹158.40

₹158.60

₹158.80

₹159.00

₹159.20

₹159.40

₹159.60

₹159.80

₹160.00

₹160.20

₹160.40

₹160.60

₹160.80

₹161.00

₹161.20

₹161.40

₹161.60

₹161.80

₹162.00

₹162.20

₹162.40

₹162.60

₹162.80

₹163.00

₹163.20

₹163.40

₹163.60

₹163.80

₹164.00

₹164.20

₹164.40

₹164.60

₹164.80

₹165.00

₹165.20

₹165.40

₹165.60

₹165.80

₹166.00

₹166.20

₹166.40

₹166.60

₹166.80

₹167.00

₹167.20

₹167.40

₹167.60

₹167.80

₹168.00

₹168.20

₹168.40

₹168.60

₹168.80

₹169.00

₹169.20

₹169.40

₹169.60

₹169.80

₹170.00

₹170.20

₹170.40

₹170.60

₹170.80

₹171.00

₹171.20

₹171.40

₹171.60

₹171.80

₹172.00

₹172.20

₹172.40

₹172.60

₹172.80

₹173.00

₹173.20

₹173.40

₹173.60

₹173.80

₹174.00

₹174.20

₹174.40

₹174.60

₹174.80

₹175.00

₹175.20

₹175.40

₹175.60

₹175.80

₹176.00

₹176.20

₹176.40

₹176.60

₹176.80

₹177.00

₹177.20

₹177.40

₹177.60

₹177.80

₹178.00

₹178.20

₹178.40

₹178.60

₹178.80

₹179.00

₹179.20

₹179.40

₹179.60

₹179.80

₹180.00

₹180.20

₹180.40

₹180.60

₹180.80

₹181.00

₹181.20

₹181.40

₹181.60

₹181.80

₹182.00

₹182.20

₹182.40

₹182.60

₹182.80

₹183.00

₹183.20

₹183.40

₹183.60

₹183.80

₹184.00

₹184.20

₹184.40

₹184.60

₹184.80

₹185.00

₹185.20

₹185.40

₹185.60

₹185.80

₹186.00

₹186.20

₹186.40

₹186.60

₹186.80

₹187.00

₹187.20

₹187.40

₹187.60

₹187.80

₹188.00

₹188.20

₹188.40

₹188.60

₹188.80

₹189.00

₹189.20

₹189.40

₹189.60

₹189.80

₹190.00

₹190.20

₹190.40

₹190.60

₹190.80

₹191.00

<

HealthWise

DashboardAssetsLoansLedgesImportVaultInsightsReportsNews

Prity Sharma

LightSign out

Loans

HDFC Bank home

ACTIVE

FLUATING

8.05% · 240 months · from 01 Jul 2019

MONTHLY EMI

₹45,621.69

OUTSTANDING

₹42.20 L

TOTAL INTEREST

₹57.49 L

₹29.89 L paid so far

TOTAL REPAYMENT

₹1.09 Cr

on ₹52.00 L borrowed

87 of 240 installments paid

18.8% of principal repaid

AMORTISATION SCHEDULE

Show all 240

#	DUE	EMI	PRINCIPAL	INTEREST	BALANCE	STATUS	
82	01 May 2026	₹45,621.69	₹14,561.32	₹31,060.37	₹42,94,391	PAID	01 May 2026
83	01 Jun 2026	₹45,621.69	₹14,666.29	₹30,955.40	₹42,79,725	PAID	01 Jun 2026
84	01 Jul 2026	₹45,621.69	₹14,772.01	₹30,849.68	₹42,64,953	PAID	01 Jul 2026
85	01 Aug 2026	₹45,621.69	₹14,878.49	₹30,743.20	₹42,50,074	PAID	01 Aug 2026
86	01 Sept 2026	₹45,621.69	₹14,985.74	₹30,635.95	₹42,35,089	PAID	01 Sept 2026
87	01 Oct 2026	₹45,621.69	₹15,093.76	₹30,527.93	₹42,19,995	PAID	01 Oct 2026
88	01 Nov 2026	₹45,621.69	₹15,202.56	₹30,419.13	₹42,04,792	PENDING	Mark paid
89	01 Dec 2026	₹45,621.69	₹15,312.15	₹30,309.54	₹41,89,480	PENDING	Mark paid
90	01 Jan 2027	₹45,621.69	₹15,422.52	₹30,199.17	₹41,74,058	PENDING	Mark paid
91	01 Feb 2027	₹45,621.69	₹15,533.69	₹30,088.80	₹41,58,524	PENDING	Mark paid
92	01 Mar 2027	₹45,621.69	₹15,645.66	₹29,976.83	₹41,42,870	PENDING	Mark paid

Marking an installment paid runs one transaction, the installment flips to paid, a loan entry is written to the ledger, and the loan auto-closes when the last installment clears. The net worth snapshot reflects via trigger.

PREPAYMENT SIMULATOR

LOAN TYPE (F)

PAID FROM

Nifty 50 Index Fund (UTI) · ₹9.13 L

Simulate

Accounting ₹42,19,994.91

The asset is added in the same transaction.

Today: 133 installments of ₹45,621.69 left, ₹27,88,126 interest still to pay, ending 01 Jul 2029.

Keep the EMI, finish sooner

Usually saves the most interest.

Interest saved

₹1,36,463

New EMI

₹45,621.69

Installments left

143 (-123)

Loan ends

01 Jul 2028

Apply this option

Keep the end date, lower the EMI

Press up monthly cash flow.

Interest saved

₹1,36,463

New EMI

₹45,621.69

Installments left

143 (-123)

Loan ends

01 Jul 2028

Apply this option

Simulated by FY, simulate prepayment without changing anything. Applying new CALL to apply prepayment: one transaction that ticks the loan, debits the source asset, rebalances the unpaid schedule and records the payment. Net worth does not move — the liability and the asset left by the same amount.

Amortisation schedule + prepayment simulator

HealthWise

DashboardAssetsLoansLedgesImportVaultInsightsReportsNews

Prity Sharma

LightSign out

Ledger

INCOME (SHOWN PERIOD)

₹0.00

EXPENSE (SHOWN PERIOD)

₹29,691.00

NET

-₹29,691.00

SEARCH THE LEDGER

FULL-TEXT SEARCH

pharmacy

Search

Clear

Matches word stems against a generated tokenizer column (description weighted above category), backed by a GIN index — not a L200 “%A” scan. Disabled phrases and / - operators work.

RECORD AN ENTRY

TYPE

AMOUNT (₹)

DATE

CATEGORY

DESCRIPTION

optional

Add entry

Expense

18/08/2026

Onceries

ENTRIES (13)

All types

DATE	TYPE	CATEGORY	DESCRIPTION	SOURCE	AMOUNT	
18 Oct 2026	EXPENSE	Healthcare	Apollo Pharmacy	manual	-₹2,073.00	Del
18 Sept 2026	EXPENSE	Healthcare	Apollo Pharmacy	manual	-₹2,014.00	Del
18 Aug 2026	EXPENSE	Healthcare	Apollo Pharmacy	manual	-₹2,913.00	Del
01 Jul 2026	EXPENSE	Healthcare	Apollo Pharmacy	manual	-₹2,801.00	Del
13 Jun 2026	EXPENSE	Healthcare	Apollo Pharmacy	manual	-₹2,469.00	Del
17 May 2026	EXPENSE	Healthcare	Apollo Pharmacy	manual	-₹2,657.00	Del
15 Apr 2026	EXPENSE	Healthcare	Apollo Pharmacy	manual	-₹2,436.00	Del
21 Mar 2026	EXPENSE	Healthcare	Apollo Pharmacy	manual	-₹2,567.00	Del
25 Feb 2026	EXPENSE	Healthcare	Apollo Pharmacy	manual	-₹2,932.00	Del
07 Jan 2026	EXPENSE	Healthcare	Apollo Pharmacy	manual	-₹2,188.00	Del
15 Dec 2025	EXPENSE	Healthcare	Apollo Pharmacy	manual	-₹2,444.00	Del
21 Nov 2025	EXPENSE	Healthcare	Apollo Pharmacy	manual	-₹2,263.00	Del

Ledger with full-text search

HealthWise

DashboardAssetsLoansLedgesImportVaultInsightsReportsNews

Prity Sharma

LightSign out

Bank statement (CSV)

Receipt (photo)

Import bank statement

Export a CSV from your net banking. HDFC, ICICI, SBI, Axis and Kotak layouts are detected automatically — columns are matched by name, not by position.

UPLOAD

STATEMENT FILE (CSV, MAX 1MB)

Choose file

hdfc.csv

Importing

Nothing to save and you review and confirm. Rows you already have are detected and unlinked automatically.

Statement import — auto-categorised preview

HealthWise

DashboardAssetsLoansLedgesImportVaultInsightsReportsNews

Prity Sharma

LightSign out

Bank statement (CSV)

Receipt (photo)

SCAN A RECEIPT

PHOTO OF THE RECEIPT (JPEG, PNG OR WEBP, MAX 5MB)

Choose file

receipt.png

Read receipt

Not to read on the HealthWise server with "Receipt"; this image is not sent to any outside service. You check every field before anything is saved.

RECEIPT

APOLLO PHARMACY

Pharmacy, 4th Floor, Bangalore

02/26, 25A8CA1234F125

NEW PRODUCT

BILL No: AP-72852

DATE: 14/08/2026 TIME: 18:42

Perceived: 658 c2 54.00

Items: 93.400 c1 100.00

Co-pay: 30000 140.00

Barcode: 011 20.00

Sub Total: 426.58

GST IN 12.89

GST PA 12.89

Gross Total: 452.26

Thank you! Get well soon.

Recognized text: 94% confidence

CHECK AND CONFIRM

MERCHANT

APOLLO PHARMACY

AMOUNT (₹)

455.28

DATE

09/14/2026

Source from this page line

CATEGORY

Healthcare

GSTIN

25A8CA1234F125

Keep the receipt image in my vault

Save expense of ₹455.28

Discard

Receipt OCR — fields extracted for confirmation

WEALTHWISE SCORE

854 / 900 Excellent

Payment history

100% — weight: 30%

Debt against assets

92% — weight: 25%

Number of active loans

100% — weight: 15%

Length of history

100% — weight: 15%

Loans closed cleanly

100% — weight: 15%

200 of 200 (100%) on time — 8 months — 87 months of history

Derived from WealthWise repayment records only. Not a credit bureau score. It is recomputed by database triggers whenever an EMI is paid or a loan changes.

SCORE OVER THE LAST YEAR

900

850

800

750

700

650

600

550

500

450

400

350

300

250

200

150

100

50

0

Oct 25

Dec 25

Feb 26

Apr 26

Jun 26

Aug 26

Oct 26

The shared band is 750 and above. Each point is re-evaluated using only the data that existed on that date.

FIND A LOAN

LOAN TYPE

AMOUNT (₹)

TENURE (MONTHS)

Home

2500000

240

Compare lenders

LENDER	EST. RATE	EMI	FEE	TOTAL COST	FOIR	MATCH	STATUS
LIC Housing	8.66%	₹21,949	₹5,258	₹52.74 L	49.3%	79	ELIGIBLE
SBI	8.53%	₹21,962	₹5,758	₹52.65 L	49.5%	79	ELIGIBLE
HDFC Bank	8.59%	₹21,838	₹12,568	₹52.54 L	49.5%	78	ELIGIBLE
ICICI Bank	9.03%	₹22,541	₹12,508	₹54.22 L	49.8%	74	ELIGIBLE

Rates are estimated within each lender's published band from your score. FOIR is all year EMI, including this one, as a share of average monthly income. Lenders usually cap it near 50%. Ranked by fit. Recommended: 1-star products, 40% rate, 25% score threshold, 20% affordability, 15% fee. Indicator only — no lender has been contacted.

WHAT IF I CLOSED A LOAN EARLY?

CLOSE THIS LOAN TODAY

ASSET GROWTH (% A YEAR)

HORIZON (YEARS)

HDFC Bank - Home - ₹42.20 L

8

5 years

In 5 years, net worth would be **+₹1.49 L** compared with keeping the loan (FI.98 Cr = ₹1.99 Cr)

NET WORTH GAINED BY CLOSING IT NOW, MONTH BY MONTH

₹1.49

₹1.48

₹1.47

₹1.46

₹1.45

₹1.44

₹1.43

₹1.42

₹1.41

₹1.40

₹1.39

₹1.38

₹1.37

₹1.36

₹1.35

₹1.34

₹1.33

₹1.32

₹1.31

₹1.30

₹1.29

₹1.28

₹1.27

₹1.26

₹1.25

₹1.24

₹1.23

₹1.22

₹1.21

₹1.20

₹1.19

₹1.18

₹1.17

₹1.16

₹1.15

₹1.14

₹1.13

₹1.12

₹1.11

₹1.10

₹1.09

₹1.08

₹1.07

₹1.06

₹1.05

₹1.04

₹1.03

₹1.02

₹1.01

₹1.00

₹0.99

₹0.98

₹0.97

₹0.96

₹0.95

₹0.94

₹0.93

₹0.92

₹0.91

₹0.90

₹0.89

₹0.88

₹0.87

₹0.86

₹0.85

₹0.84

₹0.83

₹0.82

₹0.81

₹0.80

₹0.79

₹0.78

₹0.77

₹0.76

₹0.75

₹0.74

₹0.73

₹0.72

₹0.71

₹0.70

₹0.69

₹0.68

₹0.67

₹0.66

₹0.65

₹0.64

₹0.63

₹0.62

₹0.61

₹0.60

₹0.59

₹0.58

₹0.57

₹0.56

₹0.55

₹0.54

₹0.53

₹0.52

₹0.51

₹0.50

₹0.49

₹0.48

₹0.47

₹0.46

₹0.45

₹0.44

₹0.43

₹0.42

₹0.41

₹0.40

₹0.39

₹0.38

₹0.37

₹0.36

₹0.35

₹0.34

₹0.33

₹0.32

₹0.31

₹0.30

₹0.29

₹0.28

₹0.27

₹0.26

₹0.25

₹0.24

₹0.23

₹0.22

₹0.21

₹0.20

₹0.19

₹0.18

₹0.17

₹0.16

₹0.15

₹0.14

₹0.13

₹0.12

₹0.11

₹0.10

₹0.09

₹0.08

₹0.07

₹0.06

₹0.05

₹0.04

₹0.03

₹0.02

₹0.01

₹0.00

₹-0.01

₹-0.02

₹-0.03

₹-0.04

₹-0.05

₹-0.06

₹-0.07

₹-0.08

₹-0.09

₹-0.10

₹-0.11

₹-0.12

₹-0.13

₹-0.14

₹-0.15

₹-0.16

₹-0.17

₹-0.18

₹-0.19

₹-0.20

₹-0.21

₹-0.22

₹-0.23

₹-0.24

₹-0.25

₹-0.26

₹-0.27

₹-0.28

₹-0.29

₹-0.30

₹-0.31

₹-0.32

₹-0.33

₹-0.34

₹-0.35

₹-0.36

₹-0.37

₹-0.38

₹-0.39

₹-0.40

₹-0.41

₹-0.42

₹-0.43

₹-0.44

₹-0.45

₹-0.46

₹-0.47

₹-0.48

₹-0.49

₹-0.50

₹-0.51

₹-0.52

₹-0.53

₹-0.54

₹-0.55

₹-0.56

₹-0.57

₹-0.58

₹-0.59

₹-0.60

₹-0.61

₹-0.62

₹-0.63

₹-0.64

₹-0.65

₹-0.66

₹-0.67

₹-0.68

₹-0.69

₹-0.70

₹-0.71

₹-0.72

₹-0.73

₹-0.74

₹-0.75

₹-0.76

₹-0.77

₹-0.78

₹-0.79

₹-0.80

₹-0.81

₹-0.82

₹-0.83

₹-0.84

₹-0.85

₹-0.86

₹-0.87

₹-0.88

₹-0.89

₹-0.90

₹-0.91

₹-0.92

₹-0.93

₹-0.94

₹-0.95

₹-0.96

₹-0.97

₹-0.98

₹-0.99

₹-1.00

₹-1.01

₹-1.02

₹-1.03

₹-1.04

₹-1.05

₹-1.06

₹-1.07

₹-1.08

₹-1.09

₹-1.10

₹-1.11

₹-1.12

₹-1.13

₹-1.14

₹-1.15

₹-1.16

₹-1.17

₹-1.18

₹-1.19

₹-1.20

₹-1.21

₹-1.22

₹-1.23

₹-1.24

₹-1.25

₹-1.26

₹-1.27

₹-1.28

₹-1.29

₹-1.30

₹-1.31

₹-1.32

₹-1.33

₹-1.34

₹-1.35

₹-1.36

₹-1.37

₹-1.38

₹-1.39

₹-1.40

₹-1.41

₹-1.42

₹-1.43

₹-1.44

₹-1.45

₹-1.46

₹-1.47

₹-1.48

₹-1.49

₹-1.50

₹-1.51

₹-1.52

₹-1.53

₹-1.54

₹-1.55

₹-1.56

₹-1.57

₹-1.58

₹-1.59

₹-1.60

₹-1.61

₹-1.62

₹-1.63

₹-1.64

₹-1.65

₹-1.66

₹-1.67

₹-1.68

₹-1.69

₹-1.70

₹-1.71

₹-1.72

₹-1.73

₹-1.74

₹-1.75

₹-1.76

₹-1.77

₹-1.78

₹-1.79

₹-1.80

₹-1.81

₹-1.82

₹-1.83

₹-1.84

₹-1.85

₹-1.86

₹-1.87

₹-1.88

₹-1.89

₹-1.90

₹-1.91

₹-1.92

₹-1.93

₹-1.94

₹-1.95

₹-1.96

₹-1.97

₹-1.98

₹-1.99

₹-2.00

₹-2.01

₹-2.02

₹-2.03

₹-2.04

₹-2.05

₹-2.06

₹-2.07

₹-2.08

₹-2.09

₹-2.10

₹-2.11

₹-2.12

₹-2.13

₹-2.14

₹-2.15

₹-2.16

₹-2.17

₹-2.18

₹-2.19

₹-2.20

₹-2.21

₹-2.22

₹-2.23

₹-2.24

₹-2.25

₹-2.26

₹-2.27

₹-2.28

₹-2.29

₹-2.30

₹-2.31

₹-2.32

₹-2.33

₹-2.34

₹-2.35

₹-2.36

₹-2.37

₹-2.38

₹-2.39

₹-2.40

₹-2.41

₹-2.42

₹-2.43

₹-2.44

₹-2.45

₹-2.46

₹-2.47

₹-2.48

₹-2.49

₹-2.50

₹-2.51

₹-2.52

₹-2.53

₹-2.54

₹-2.55

₹-2.56

₹-2.57

₹-2.58

₹-2.59

₹-2.60

₹-2.61

₹-2.62

₹-2.63

₹-2.64

₹-2.65

₹-2.66

₹-2.67

₹-2.68

₹-2.69

₹-2.70

₹-2.71

₹-2.72

₹-2.73

₹-2.74

₹-2.75

₹-2.76

₹-2.77

₹-2.78

₹-2.79

₹-2.80

₹-2.81

₹-2.82

₹-2.83

₹-2.84

₹-2.85

₹-2.86

₹-2.87

₹-2.88

₹-2.89

₹-2.90

₹-2.91

₹-2.92

₹-2.93

₹-2.94

₹-2.95

₹-2.96

₹-2.97

₹-2.98

₹-2.99

₹-3.00

₹-3.01

₹-3.02

₹-3.03

₹-3.04

₹-3.05

₹-3.06

₹-3.07

₹-3.08

₹-3.09

₹-3.10

₹-3.11

₹-3.12

₹-3.13

₹-3.14

₹-3.15

₹-3.16

₹-3.17

₹-3.18

₹-3.19

₹-3.20

₹-3.21

₹-3.22

₹-3.23

₹-3.24

₹-3.25

₹-3.26

₹-3.27

₹-3.28

₹-3.29

₹-3.30

₹-3.31

₹-3.32

₹-3.33

₹-3.34

₹-3.35

₹-3.36

₹-3.37

₹-3.38

₹-3.39

₹-3.40

₹-3.41

₹-3.42

₹-3.43

₹-3.44

₹-3.45

₹-3.46

₹-3.47

₹-3.48

₹-3.49

₹-3.50

₹-3.51

₹-3.52

₹-3.53

₹-3.54

₹-3.55

₹-3.56

₹-3.57

₹-3.58

₹-3.59

₹-3.60

₹-3.61

₹-3.62

₹-3.63

₹-3.64

₹-3.65

₹-3.66

₹-3.67

₹-3.68

₹-3.69

₹-3.70

₹-3.71

₹-3.72

₹-3.73

₹-3.74

₹-3.75

₹-3.76

₹-3.77

₹-3.78

₹-3.79

₹-3.80

₹-3.81

₹-3.82

₹-3.83

₹-3.84

₹-3.85

₹-3.86

₹-3.87

₹-3.88

₹-3.89

₹-3.90

₹-3.91

₹-3.92

₹-3.93

₹-3.94

₹-3.95

₹-3.96

₹-3.97

₹-3.98

₹-3.99

₹-4.00

₹-4.01

₹-4.02

₹-4.03

₹-4.04

₹-4.05

₹-4.06

₹-4.07

₹-4.08

₹-4.09

₹-4.10

₹-4.11

₹-4.12

₹-4.13

₹-4.14

₹-4.15

₹-4.16

₹-4.17

₹-4.18

₹-4.19

₹-4.20

₹-4.21

₹-4.22

₹-4.23

₹-4.24

₹-4.25

₹-4.26

₹-4.27

₹-4.28

₹-4.29

₹-4.30

₹-4.31

₹-4.32

₹-4.33

₹-4.34

₹-4.35

₹-4.36

₹-4.37

₹-4.38

₹-4.39

₹-4.40

₹-4.41

₹-4.42

₹-4.43

₹-4.44

₹-4.45

₹-4.46

₹-4.47

₹-4.48

₹-4.49

₹-4.50

₹-4.51

₹-4.52

₹-4.53

₹-4.54

₹-4.55

₹-4.56

₹-4.57

₹-4.58

₹-4.59

₹-4.60

₹-4.61

₹-4.62

₹-4.63

₹-4.64

₹-4.65

₹-4.66

₹-4.67

₹-4.68

₹-4.69

₹-4.70

₹-4.71

₹-4.72

₹-4.73

₹-4.74

₹-4.75

₹-4.76

₹-4.77

₹-4.78

₹-4.79

₹-4.80

₹-4.81

₹-4.82

₹-4.83

₹-4.84

₹-4.85

₹-4.86

₹-4.87

₹-4.88

₹-4.89

₹-4.90

₹-4.91

₹-4.92

₹-4.93

₹-4.94

₹-4.95

₹-4.96

₹-4.97

₹-4.98

₹-4.99

₹-5.00

₹-5.01

₹-5.02

₹-5.03

₹-5.04

₹-5.05

₹-5.06

₹-5.07

₹-5.08

₹-5.09

₹-5.10

₹-5.11

₹-5.12

₹-5.13

₹-5.14

₹-5.15

₹-5.16

₹-5.17

₹-5.18

₹-5.19

₹-5.20

₹-5.21

₹-5.22

₹-5.23

₹-5.24

₹-5.25

₹-5.26

₹-5.27

₹-5.28

₹-5.29

₹-5.30

₹-5.31

₹-5.32

₹-5.33

₹-5.34

₹-5.35

₹-5.36

₹-5.37

₹-5.38

₹-5.39

₹-5.40

₹-5.41

₹-5.42

₹-5.43

₹-5.44

₹-5.45

₹-5.46

₹-5.47

₹-5.48

₹-5.49

₹-5.50

₹-5.51

₹-5.52

₹-5.53

₹-5.54

₹-5.55

₹-5.56

₹-5.57

₹-5.58

₹-5.59

₹-5.60

₹-5.61

₹-5.62

₹-5.63

₹-5.64

₹-5.65

₹-5.66

₹-5.67

₹-5.68

₹-5.69

₹-5.70

₹-5.71

₹-5.72

₹-5.73

₹-5.74

₹-5.75

₹-5.76

₹-5.77

₹-5.78

₹-5.79

₹-5.80

₹-5.81

₹-5.82

₹-5.83

₹-5.84

₹-5.85

₹-5.86

₹-5.87

₹-5.88

₹-5.89

₹-5.90

₹-5.91

₹-5.92

₹-5.93

₹-5.94

₹-5.95

₹-5.96

₹-5.97

₹-5.98

₹-5.99

₹-6.00

₹-6.01

₹-6.02

₹-6.03

₹-6.04

₹-6.05

₹-6.06

₹-6.07

₹-6.08

₹-6.09

₹-6.10

₹-6.11

₹-6.12

₹-6.13

₹-6.14

₹-6.15

₹-6.16

₹-6.17

₹-6.18

₹-6.19

₹-6.20

₹-6.21

₹-6.22

₹-6.23

₹-6.24

₹-6.25

₹-6.26

₹-6.27

₹-6.28

₹-6.29

₹-6.30

₹-6.31

₹-6.32

₹-6.33

₹-6.34

₹-6.35

₹-6.36

₹-6.37

₹-6.38

₹-6.39

₹-6.40

₹-6.41

₹-6.42

₹-6.43

₹-6.44

₹-6.45

₹-6.46

₹-6.47

₹-6.48

₹-6.49

₹-6.50

₹-6.51

₹-6.52

₹-6.53

₹-6.54

₹-6.55

₹-6.56

₹-6.57

₹-6.58

₹-6.59

₹-6.60

₹-6.61

₹-6.62

₹-6.63

₹-6.64

₹-6.65

₹-6.66

₹-6.67

₹-6.68

₹-6.69

₹-6.70

₹-6.71

₹-6.72

₹-6.73

₹-6.74

₹-6.75

₹-6.76

₹-6.77

₹-6.78

₹-6.79

₹-6.80

₹-6.81

₹-6.82

₹-6.83

₹-6.84

₹-6.85

₹-6.86

₹-6.87

₹-6.88

₹-6.89

₹-6.90

₹-6.91

₹-6.92

₹-6.93

₹-6.94

₹-6.95

₹-6.96

₹-6.97

₹-6.98

₹-6.99

₹-7.00

₹-7.01

₹-7.02

₹-7.03

₹-7.04

₹-7.05

₹-7.06

₹-7.07

₹-7.08

₹-7.09

₹-7.10

₹-7.11

₹-7.12

₹-7.13

₹-7.14

₹-7.15

₹-7.16

₹-7.17

₹-7.18

₹-7.19

₹-7.20

₹-7.21

₹-7.22

₹-7.23

₹-7.24

₹-7.25

₹-7.26

₹-7.27

₹-7.28

₹-7.29

₹-7.30

₹-7.31

₹-7.32

₹-7.33

₹-7.34

₹-7.35

₹-7.36

₹-7.37

₹-7.38

₹-7.39

₹-7.40

₹-7.41

₹-7.42

₹-7.43

₹-7.44

₹-7.45

₹-7.46

₹-7.47

₹-7.48

₹-7.49

₹-7.50

₹-7.51

₹-7.52

₹-7.53

₹-7.54

₹-7.55

₹-7.56

₹-7.57

₹-7.58

₹-7.59

₹-7.60

₹-7.61

₹-7.62

₹-7.63

₹-7.64

₹-7.65

₹-7.66

₹-7.67

₹-7.68

₹-7.69

₹-7.70

₹-7.71

₹-7.72

₹-7.73

₹-7.74

₹-7.75

₹-7.76

₹-7.77

₹-7.78

₹-7.79

₹-7.80

₹-7.81

₹-7.82

₹-7.83

₹-7.84

₹-7.85

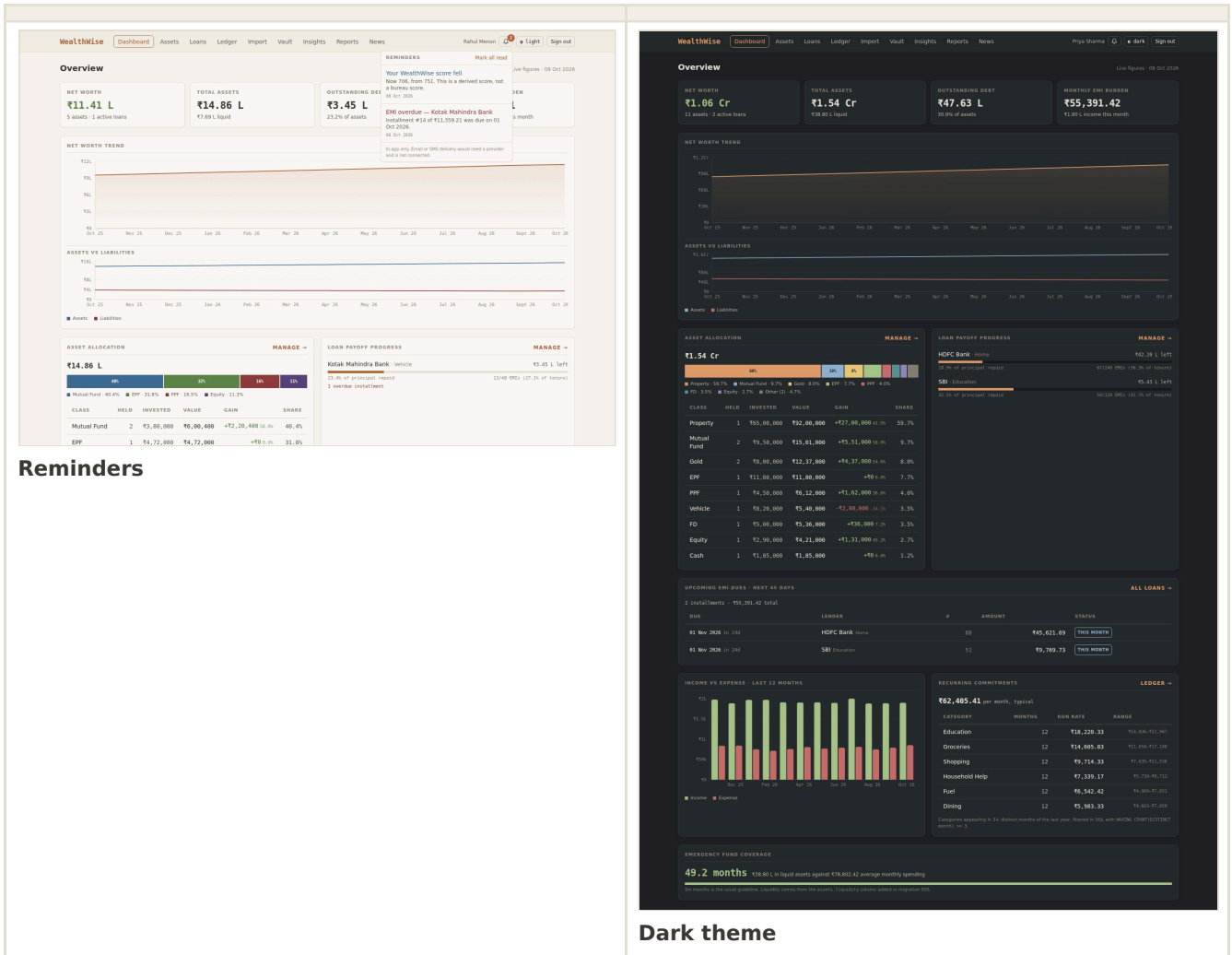
₹-7.86

₹-7.87

₹-7.88

₹-7.89

₹-7.9



13. Testing

13.1 Automated suite

```
npm test          # = npm run test:unit && npm run test:db
```

Database tests run against a separate `wealthwise_test` database, rebuilt from the migrations before every run, so they never touch demo data. Each test file creates its own users, so files share no state.

Suite	Tests	Covers
CSV reader, dates, amounts, header detection, normalisation	17	Quoted commas, Indian grouping, <code>31/02</code> rejection, dedupe hash
Receipt extraction, news classification, RSS parsing	11	Grand Total over Sub Total, month-name dates, rate-change direction
Integrity constraints	5	Negative amounts, rates > 60%, bad PAN, paid-without-date, duplicate email
EMI functions	4	Formula value, zero-interest case, exact principal sum, schedule lock
Atomic EMI payment	4	Ledger + outstanding, double-pay refused, cross-user refused, auto-close

Suite	Tests	Covers
Net worth trigger	2	Asset and loan inserts refresh the snapshot
Statement import	4	Categorisation, re-import inserts 0, mid-batch failure rolls back everything , revert
Categorisation rules	3	User rule beats system rule; duplicate system rule refused; retroactive sweep
Full-text search	2	Stemming; planner uses the GIN index
Credit score	3	Range and disclaimer; overdue lowers it; statement trigger fires once for 24 rows
Prepayment	5	Tenure mode beats EMI mode; over-payment refused; net worth unchanged; under-funded rolls back ; cross-user refused
Recommendations	1	Ineligible below the minimum score, with reason; eligible ranked first
What-if projection	1	At 0% growth, the gain equals the remaining interest exactly
Financial year	2	31 Mar vs 1 Apr boundary; view splits correctly
Views	2	HAVING keeps only 3+ month categories; zero row for an empty user
Reminders	1	Generated once, idempotent on re-run
Total	67	67 passed, 0 failed

Full output, test by test: [docs/test-results.txt](#).

13.2 End-to-end verification

Beyond the automated suite, every API endpoint was exercised over HTTP against the running application, and every page was checked by screenshot in both themes. Selected results:

Check	Result
₹5L prepayment on the seeded ₹52L home loan, seven years in	Saves ₹8.51L interest and 29 months (reduce tenure) vs ₹3.27L (reduce EMI)
Same prepayment funded from an FD	Net worth unchanged to the paise; FD and liability both fall ₹3L
Edit a loan that has payments	HTTP 409, with a hint to use prepayment instead
Pay or prepay another user's loan	404 / no change
Individual calls the advisor API	403
Re-import the same bank statement	0 inserted, 10 reported as duplicates
Confirm the same receipt twice	201, then 409
Receipt OCR on a pharmacy bill	94% confidence, 1.5 s; merchant, ₹455.28, date and GSTIN all correct
+25 bps repo rate on the seeded user	EMIs +₹665.37 a month, about ₹96,295 over the remaining tenure
PDF statement	2 pages, embedded fonts, 83 ₹ glyphs rendered, PAN masked

13.3 Defects found by testing, and fixed

Testing found real defects, not only confirmed working code:

1. `UPDATE ... FROM LATERAL` **referencing the update target**. Postgres rejects this; the original `fn_recategorize_user` failed on first use. Replaced with a correlated subquery in migration 007.
2. **Prepayment created net worth from nothing**. The first version lowered the liability without debiting the cash that paid it, so net worth jumped by the full prepayment. The procedure now debits a named source asset in the same transaction, and a test asserts net worth is unchanged.
3. **Projection overstated the benefit**. It assumed every remaining EMI equalled the next one, which is wrong after a reduce-tenure prepayment (the last EMI is smaller). It now sums the actual schedule; the 0%-growth test pins it exactly.
4. **Editing a paid loan would erase its payment history**. Schedule regeneration deleted every row. It is now refused at the database level (`WW409`).
5. **Form validation blocked valid input**. `min=1 step=1000` made ₹25,00,000 invalid in the browser, so the lender comparison never submitted. Found by screenshot review.
6. **PDF spilled onto blank pages**. Footers drawn inside the bottom margin triggered page breaks (6 pages instead of 2).
7. **Charts**. Two asset classes shared one grey; an axis printed "₹2L" twice; "₹50k" rendered as "₹5k"; and a two-line projection chart was unreadable at scale, so it was redrawn to plot the difference.

13.4 Chart accessibility

Chart colours were checked with a colour-vision-deficiency validator, not chosen by eye. The seven-hue categorical palette measures ΔE 10.1 / 9.8 (dark / light) under deuteranopia simulation and 18.3 for normal vision, with contrast passing in both themes. Every chart also has a legend, direct labels or a data table, so colour is never the only channel.

14. Conclusion and future enhancements

WealthWise shows that a personal-finance application is a strong test of database design. The domain needs exact arithmetic, strict integrity and atomic multi-table updates, which suit a relational database, alongside irregular documents and feeds, which suit a document store. Putting the financial logic in the database (amortisation in a function, prepayment in a procedure, net worth and scores maintained by triggers, reporting in views) made it explainable and testable: the same EMI figure appears on the schedule, the dashboard, the PDF and the news impact, because one function computes all of them.

Future enhancements

- **Account Aggregator integration** (RBI AA framework) to replace manual CSV upload with consented bank data.
- **Partitioning** `transactions` by financial year (`PARTITION BY RANGE`) once volumes grow; the expression index already aligns with that key.
- **Row-level security** policies, so tenant isolation is enforced by Postgres itself, in addition to the application's `WHERE user_id = ...` clauses.
- **Materialised views** with scheduled refresh for advisor-wide analytics over many clients.

- **Semantic search over documents and news** using `pgvector`, should unstructured content grow enough to warrant it.
- **Email and SMS reminders** through a delivery provider, and a real scheduler in place of refresh-on-read.
- **Cloud OCR** as an opt-in fallback for low-confidence photographed receipts.
- **Floating-rate resets** applied automatically from the news feed to repo-linked loans.

15. Individual contributions

To be completed by the team. The brief (§2, §15, §16) requires each member's contribution to be visible in the commit history. Fill in this table, and make sure each member commits their own work under their own Git identity.

Area	Member	Key files
Relational schema, constraints, indexes		<code>db/migrations/001</code> , <code>002</code>
EMI functions, triggers, views		<code>003</code> , <code>004</code> , <code>005</code>
Import, categorisation, full-text search		<code>006</code> , <code>src/lib/services/csv-parser.ts</code> , <code>src/lib/db/imports.ts</code>
Smart features (prepayment, score, recommendations, projection)		<code>008</code> , <code>src/lib/db/insights.ts</code>
MongoDB model and aggregation (vault, imports, news)		<code>src/lib/db/mongo.ts</code> , <code>src/lib/services/*-service.ts</code>
API layer, auth and roles		<code>src/app/api/**</code> , <code>src/lib/auth/**</code>
UI, design system, charts		<code>src/components/**</code> , <code>src/app/globals.css</code>
OCR and PDF statement		<code>ocr-service.ts</code> , <code>statement-pdf.ts</code>
Tests		<code>tests/**</code>
Documentation		<code>README.md</code> , <code>docs/**</code>